Sentinel-RX
Security-Aware Agentic Recovery of Failing Systems

Technical Report — Humain Hackathon, Challenge 1: Agentic Recovery of a Failing System

Abdulrahman Alharbi

30 October 2026

Summary. Sentinel-RX is an agent that identifies, diagnoses, recovers and *verifies* the recovery of a failing enterprise system in which several faults are active at once and a security compromise may hide inside an operational incident. Four cheap deterministic detectors (robust metric z-scores, log-template signatures, a dependency-aware root-cause ranker and a RandomForest beacon detector) compress thousands of telemetry records into a short ranked evidence list; a local 7B model (`qwen2.5:7b` via Ollama) reasons over it inside a MAPE-K loop with a step budget; a safety layer enforces an action allowlist, contain-before-restart for compromised hosts and transactional rollback; and a verification gate permits a "recovered" claim only when post-action telemetry matches the pre-incident baseline and no beacon attempt remains. On 20 held-out incidents never used during development (3 seeds, $n = 60$) the agent recovers 100 % of systems with 0 % false claims and zero harmful actions, against 45 % for a rule-based runbook and 30 % for a single-shot LLM that falsely claims recovery 70 % of the time. The benchmark, the simulator, a live five-process environment, a replay cache that makes every number reproducible without an LLM, and 109 tests ship with the code. Everything ran on one laptop with no cloud and no Docker.

Repository: `github.com/AbdulAlharbi/sentinel-rx` (reproduction commands in Appendix A). Literature review: `docs/literature_review.pdf`.

Executive summary

Problem. Automated incident recovery is evaluated almost exclusively on single, accidental faults. Real incidents involve several interacting faults, symptoms that propagate along service dependencies, and sometimes a compromise that looks like a performance problem. An agent that restarts a compromised host destroys evidence and lets the implant re-establish; an agent that declares "recovered" without checking is worse than no agent at all.

What was built. A complete, locally reproducible system and benchmark:

- a deterministic simulator of a five-service enterprise with dependency propagation, nine known fault types plus one held-out unseen type, and hidden ground truth;
- a live environment of five real local processes with real memory growth, a real dead process, a real connection-pool hold and real periodic HTTP beacons to a local sinkhole;
- the agent: detectors → LLM hypotheses and targeted checks → ordered plan → safety layer → verification gate, two LLM calls per iteration;
- a benchmark with dev/held-out split enforcement, perturbations, six baselines and sanity floors, four ablations, bootstrap confidence intervals, and a mid-development scorer audit.

Headline result (held-out, `qwen2.5:7b`, 20 scenarios × 3 seeds).

configuration	top-1 root cause	recovered	false "recovered"	harmful / run	LLM tokens / run
Sentinel-RX agent	92 %	100 %	0 %	0.00	6 103
runbook (rules, no LLM)	87 %	45 %	5 %	0.00	0
single-shot LLM	63 %	30 %	70 %	0.27	2 421
LLM only (no detectors, $n = 20$)	35 %	35 %	15 %	0.55	35 018
do-nothing / random floors	0 %	0 %	0 % / 10 %	0.00 / 3.45	0

Why it works. Ablations show each component protects a different metric: removing the verification gate raises false claims from 0 % to 80 %; removing the loop drops recovery to 20 %; removing the beacon detector drops recovery to 50 % and raises false claims to 35 %; removing the safety layer keeps recovery at 100 % but introduces 0.60 harmful actions per run. Under noisy logs, missing metrics, renamed services, timestamp jitter and an unseen fault type, recovery stays at 95 %–100 % (the unseen type is recovered, not identified). A 1.5B model recovers only 50 % but still makes 0 % false claims and zero harmful actions, because those guarantees live in the detectors, the safety layer and the verifier rather than in the model.

Honesty. A scorer audit during development found that early versions were *not* harmless (0.27 and 0.23 containment actions per run on uncompromised hosts hidden by the old scorer) and that the observable health formula was too lenient; both were fixed, all earlier runs were re-scored, and the do-nothing and random baselines remain in the benchmark as permanent sanity floors. Several evaluation arms use one seed because of the laptop time budget; their deltas are directional.

Contents

Executive summary **1**

1 Introduction **3**
1.1 Contributions . . . 3

2 State of the Field **3**

3 System Overview **5**

4 Implementation **5**
4.1 Environment interface . . . 5
4.2 SimEnv: tick model and health . . . 6
4.3 Fault taxonomy . . . 6
4.4 LiveEnv: five real processes . . . 7
4.5 Detectors . . . 7
4.6 LLM backend and record/replay cache . . . 9
4.7 Safety layer . . . 9

5 Agent Algorithm **11**

6 Evaluation Methodology **13**
6.1 Scenario generation and split discipline . . . 13
6.2 Perturbations . . . 14
6.3 Baselines and ablations . . . 14
6.4 Metrics . . . 14

7 Results **15**
7.1 Held-out benchmark . . . 15
7.2 Robustness . . . 16
7.3 Ablations . . . 18
7.4 Cost . . . 18
7.5 Live demonstration . . . 19

8 Scorer Audit **20**

9 Discussion **21**

10 Limitations and Threats to Validity **22**

11 Path to Production **23**

12 Conclusion **23**

A Reproduction commands **25**

B Repository map **26**

1 Introduction

Incident response is the most expensive and least automated stage of operating software. An on-call engineer must notice that something is wrong, localise it in a dependency graph where every downstream fault produces upstream symptoms, decide what to do, do it without making things worse, and confirm that the system is actually healthy again. The post-2023 literature has moved from "can a model rank likely root causes?" to "can an agent own the incident end to end?" [8, 9, 12], yet reported success rates on realistic benchmarks remain around 11 % [10, 9], and evaluations almost never address three things that make real incidents hard.

Concurrent faults. With one fault per scenario, the deepest anomalous service is the answer. With two or three, the symptom pattern is a superposition: a memory leak on `api` and a connection-pool exhaustion on `db` both show up as errors on `api` and `gateway`. Classical root-cause analysis assumes a single cause; a model that fixes one fault and sees the symptom persist must decide whether its fix was wrong or merely incomplete.

Hidden security incidents. A command-and-control (C2) beacon adds a few percent of CPU and a handful of small periodic outbound flows hidden among health checks and update polls. If an ops agent treats the host like any slow service and restarts it, it destroys forensic evidence and the implant re-establishes. The correct order is *contain, then eradicate*, and no AIOps benchmark we reviewed tests that [9, 8].

Verification. An agent that declares success is believed. A false "recovered" claim closes the incident with the fault still active, which is strictly worse than escalating. Mitigation success is measured in the literature; false recovery as a distinct error class almost never is.

1.1 Contributions

1. **A security-aware hybrid agent** (sections 3 to 5): deterministic detectors feed a local 7B model inside a budgeted MAPE-K loop; a safety layer enforces an allowlist, evidence-gated containment, contain-before-restart ordering and transactional rollback; a verification gate compares post-action telemetry with the remembered healthy baseline before any recovery claim.
2. **A reproducible benchmark** (section 6): a deterministic simulator with dependency propagation and hidden ground truth, 30 development and 20 held-out multi-fault scenarios with disjoint fault combinations, five perturbations including an unseen fault type, six baselines (two of them sanity floors), four ablations, exact metric definitions and bootstrap CIs. A content-addressed LLM cache makes every reported number replayable with no model installed.
3. **Evidence on all five literature gaps** (section 7): the agent reaches 100 % recovery, 0 % false claims and zero harmful actions on held-out incidents at 6 103 tokens per incident, while the raw-telemetry variant of the same loop needs 35 018 tokens to recover 35 %.
4. **A live environment** (section 4.4): the same agent, byte-identical, recovers a real three-fault incident on five real processes, containing the beaconing host before restarting it.
5. **An audited scorer** (section 8): the recovery metric was challenged mid-development, found lenient in two places, fixed, and all earlier runs re-scored.

2 State of the Field

The conceptual skeleton of every agentic design is still Kephart and Chess's autonomic computing vision [1]: a Monitor–Analyze–Plan–Execute loop over shared knowledge (MAPE-K). Chaos engineering [2] supplies the evaluation method: inject real faults and measure deviation from a

hypothesised steady state. Surveys of pre-LLM AIOps [3] consistently find remediation the least mature stage.

Hybrid evidence collection in production. RCACopilot [4] separates deterministic, incident-specific handlers that gather multi-source diagnostic data from an LLM that predicts the root-cause category with an explanation (Micro-F1 up to 0.766 on a year of Microsoft incidents). RCAgent [5] moves to a ReAct-style [6] controller with information-gathering tools and, importantly for this project, an *internally deployed* model plus structured-output repair that sharply reduces invalid actions. Both establish the principle Sentinel-RX follows: deterministic tooling gathers evidence, the LLM reasons over it.

Interactive benchmarks. AIOpsLab [8, 7] deploys fault-injected microservices behind an agent–cloud interface with 48 problems across detection, localisation, RCA and mitigation. ITBench [9] spans SRE, CISO and FinOps scenarios; state-of-the-art agents resolved about 11 % of SRE and 25 % of CISO scenarios. OpenRCA [10] provides 335 real failures with 68 GB of telemetry; the best agent identifies the full root cause in about 11 % of cases and often reaches correct answers through flawed reasoning. RCAEval [11] standardises AC@k over 735 cases and 11 fault types. All of them inject one fault per scenario.

Safety of autonomous remediation. STRATUS [12] organises specialised detection, diagnosis and mitigation agents in a state machine and formalises *transactional no-regression*: a mitigation that does not improve the system is rolled back. ToolEmu [13] shows that even the safest tool-using agents exhibit risky behaviour in 23.9 % of test cases. Neither addresses whether an action is *appropriate* to the nature of the incident, for example restarting a compromised container.

C2 beaconing. BAYWATCH [14] detects stealthy beaconing in enterprise proxy logs via signal processing and emphasises that legitimate software also beacons, making false positives the central challenge. CTU-13 [15] remains the standard labelled botnet dataset. The author's prior work [16] showed that periodicity features computed from flow timestamps alone (inter-arrival regularity, spectral concentration, autocorrelation, sub-window stability) rank suspicious communication on encrypted traffic (AP 0.852 vs. 0.046 for a heuristic on CTU-13 with synthetic beacon injection). These features are cheap and payload-free, which makes them usable as a deterministic detector inside an operations agent.

Gaps. The review (`docs/literature_review.pdf`) ends with five gaps that define this project's research questions:

G1 Concurrent and confounded faults.
Evaluations inject one fault; classical RCA assumes one cause.

G2 Security-blind operations.
No AIOps benchmark checks whether an agent recognises a compromise inside an operational fault, or whether its remediation is appropriate when it does.

G3 Verification of recovery.
Agents falsely declaring recovery is rarely reported as a distinct error class.

G4 Cost of the agent.
Tokens, latency and compute per incident are seldom reported; most evaluations use large proprietary models.

G5 Robustness and generalisation.
Telemetry degradation and held-out fault combinations are rarely tested systematically.

3 System Overview

Figure 1 shows the whole system. The agent never sees the simulator's fault list; it talks to an environment only through the `Env` interface (status, metrics, logs, flows, actions). Raw telemetry goes through four deterministic detectors that emit a short ranked evidence list. The LLM turns evidence into ranked hypotheses and up to three targeted checks, then into an ordered plan. The safety layer screens and reorders the plan, executes it transactionally, and the verification gate decides whether the system is healthy by comparing post-action telemetry with the healthy baseline remembered at the first observation. The C2 sinkhole stands in for the attacker's server: in the simulator it is a destination address, in the live environment a real local HTTP process.

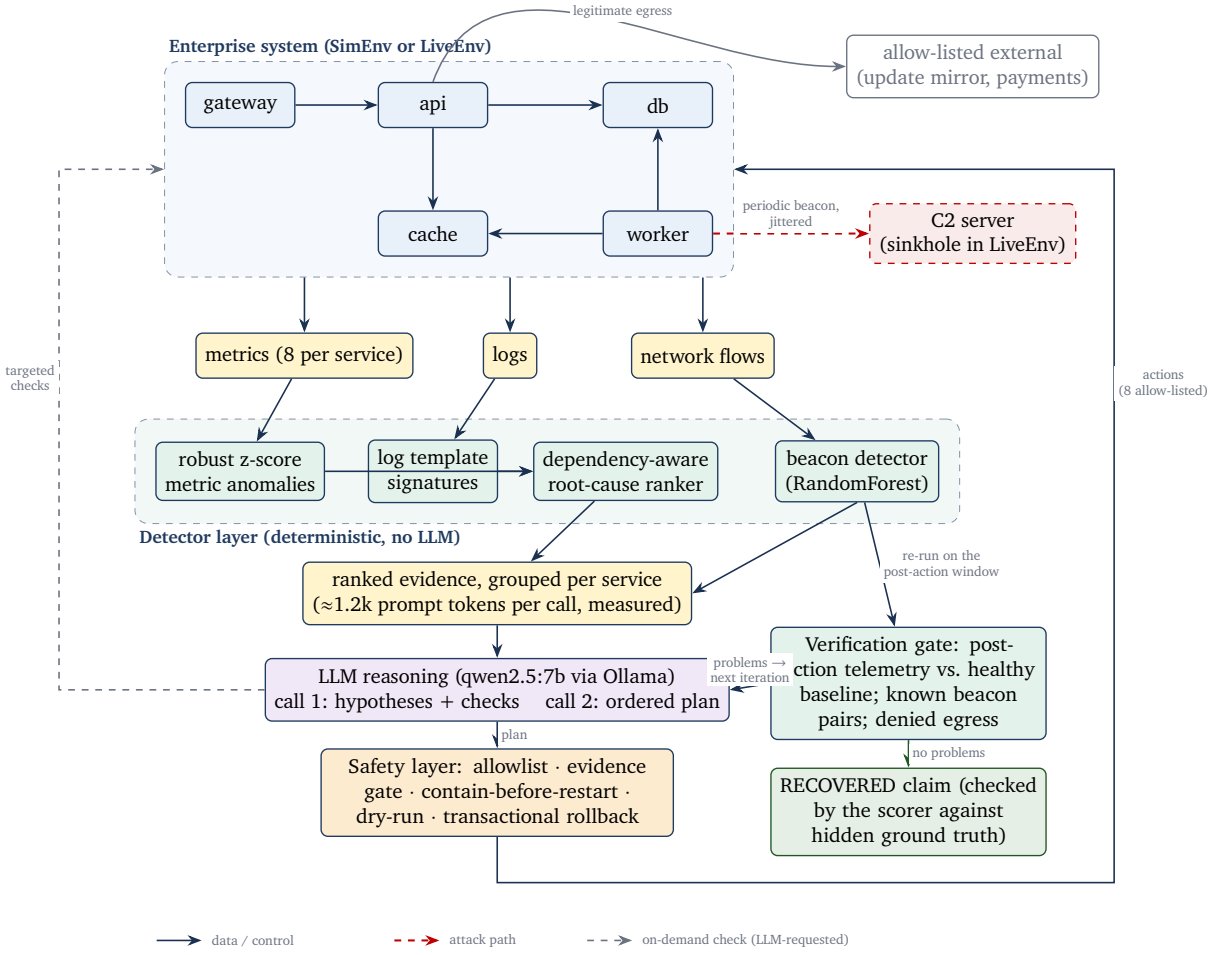


Figure 1: System architecture. Telemetry from the enterprise system is compressed by four deterministic detectors into ranked evidence; the LLM reasons, the safety layer screens and executes, and the verification gate closes the loop. The beacon path from `worker` to the C2 server is the hidden security incident.

4 Implementation

4.1 Environment interface

Both environments implement one abstract class (`env/base.py::Env`): `list_services`, `get_topology`, `get_status`, `get_metrics(window)`, `get_logs(service, window, grep)`, `get_flows(window)`, `apply_action`, `undo_action`, `wait`, `snapshot`, `now`. Everything not on the interface is invisible to the agent. The benchmark runner reaches hidden ground truth through `SimEnv.gt_*` methods wrapped in a probe that the agent cannot call.

Design decision — one agent interface, two environments

Benchmarks need thousands of cheap, seeded, reproducible episodes; a demo needs real memory growth, a real dead process and real periodic HTTP beacons. Sharing one interface means the live demo is evidence that the benchmark agent is not over-fitted to simulator artefacts: the agent and detectors are byte-identical in both.

4.2 SimEnv: tick model and health

The simulator advances in discrete ticks of $\Delta t = 10\,\text{s}$ over 150 ticks; faults start between ticks 40 and 65 and the agent starts at tick 90, so the first 30 % of its 900 s observation window is a clean baseline. An action costs 3 ticks (30 s) before its effects are visible. A full episode builds in about 25 ms, which makes the 300-plus runs of the evaluation affordable.

Each service s has a healthy baseline b_s for eight metrics (latency p50/p95, error rate, CPU, memory, connection count, disk used, queue depth). Per tick, fault effects are applied to the *intrinsic* metrics of the targeted service, and *propagated* metrics flow upstream along the dependency graph with attenuation. For a service u calling dependency d:

$$e_u \mathrel{+}= 0.7\,\max\bigl(0,\ e_d - b_d^{e}\bigr), \quad \ell_u^{50} \mathrel{+}= 0.8\,\max\bigl(0,\ \ell_d^{50} - b_d^{50}\bigr), \quad \ell_u^{95} \mathrel{+}= 0.8\,\max\bigl(0,\ \ell_d^{95} - b_d^{95}\bigr), \tag{1}$$

and if d is down or at error rate ≥ 0.99 the caller receives +0.9 error rate, +800 ms p95 and +300 ms p50 instead. Memory, disk, connection count and CPU never propagate: a dependency cannot cause them, which is exactly what the ranker exploits. Blocking the egress of a service with a legitimate external dependency (`api` calls a payments API) adds 0.3 error rate while blocked, so a wrong containment is observable and costly.

Observable health is a function of the latest error rate e_s and p95 latency ℓ_s of each service:

$$h_s = \max\bigl(0,\ 1 - \min(1, e_s)\bigr) \cdot \begin{cases} 1 & \ell_s \leq 400 \\ \max\bigl(0,\ 1 - \frac{\ell_s - 400}{2000}\bigr) & \ell_s > 400, \end{cases} \qquad H = \tfrac{1}{2}\,\overline{h} + \tfrac{1}{2}\,\min_s h_s, \tag{2}$$

with $h_s = 0$ for any service not in state `up`. The min term was added by the scorer audit (section 8): with a plain mean, one degraded service among five stayed above the 0.9 bar. Ground-truth recovery is stricter than H: every injected fault resolved, every service up, and $H \geq 0.9$ for two consecutive ticks.

4.3 Fault taxonomy

Table 1 lists the ten fault types. Nine are *known* (described to the agent in its system prompt the way a runbook would); `dns_failure` exists only in the unseen-fault perturbation set. Every fault persists until the correct remediation reaches the correct target; tests assert that no fault clears on its own in 300 ticks of inaction and that none is resolved by any of the 8 actions on any other service or by any wrong action on the right service (`tests/test_faults.py`).

Table 1: Fault library (`faults/`). Containment-first is enforced for the beacon: a restart before containment is harmful, re-establishes the implant with a new interval and is counted as a reinfection.

type	targets	observable symptoms	correct remediation
memory_leak	api, worker, gateway	memory rises monotonically; later p95 and errors climb; GC warnings	restart_service
db_conn_exhaustion	db	conn_count climbs to the pool limit; "too many connections" on dependents	scale_connections or restart
disk_full	db, worker	disk_used → 100 %; "No space left on device"; errors climb	clear_disk
latency_injection	api, db, cache	p50/p95 jump on target and dependents; retransmit warnings; no errors	restart_service
bad_config_deploy	gateway, api	step change in error rate after a "config reload" line; restart does not help	rollback_config only
process_crash	api, worker, cache	status down, CPU and connections drop to 0; dependents get connection refused	restart_service
cache_failure	cache	cache errors up, api latency up, db CPU/connections up; eviction logs	flush_cache or restart
c2_beacon	worker, api	small periodic outbound flows to an unknown external IP, slight CPU rise, no errors; hidden among health checks and update polls	**block_egress or isolate_host, *then* restart_service**
credential_stuffing	gateway	burst of "authentication failed" from many IPs; CPU and latency up; auth backend saturates	rate_limit_auth
dns_failure (unseen)	api, worker	error rate spikes on the target while dependencies look healthy but receive less traffic	rollback_config or restart

Design decision — a contained implant stays observable

After `block_egress` the implant keeps *trying*: the firewall emits flows with `blocked=True` and an "egress denied" log line. Without this, no verifier could distinguish "contained" from "eradicated" and a false "recovered" claim would be unfalsifiable. The live services do the same.

4.4 LiveEnv: five real processes

The live environment (`env/live/`) starts five stdlib `http.server` processes (`gateway` calls `api`, which calls `db` (SQLite) and `cache`; `worker` calls `db` and `cache`), a load generator, a monitor thread that samples metrics every 2 s and emits health-check flows, and a sinkhole that plays both the C2 server and the update mirror. Faults are real where that is safe (the heap actually grows, a process actually exits, the pool is actually held, periodic HTTP calls actually happen) and sandboxed where it would hurt the laptop ("disk full" is a 120 MB quota on a sandbox directory; memory pressure slows handlers in proportion to leaked bytes). A restart without containment relaunches the process *with the implant re-injected*, mirroring the simulator. Everything is torn down by `atexit` and signal handlers.

4.5 Detectors

Metric anomalies. For every (service, metric) series the detector compares the recent window with the remembered pre-incident baseline using a robust z-score,

$$z = \frac{\left|\operatorname{med}(x_{\text{recent}}) - \operatorname{med}(x_{\text{base}})\right|}{\max\left(1.4826\ \operatorname{MAD}(x_{\text{base}}),\ 0.02\,\left|\operatorname{med}(x_{\text{base}})\right|,\ 10^{-6}\right)}, \tag{3}$$

and flags the series when $z \geq 6$ and the absolute shift exceeds a per-metric floor (for example 0.05 error rate, 50 ms p95). Missing values are ignored, which is what makes the `missing_metrics` perturbation survivable.

Log signatures. Numbers and identifiers are masked to <*> to extract templates; templates that are new or surge relative to the baseline are weighted by severity. This catches the one line that names the fault ("too many connections", "No space left on device", "authentication failed").

Dependency-aware ranker. Let I_s be the intrinsic anomaly strength of service s (maximum over intrinsic-metric z-scores scaled by 30, log-signature score, and 1 if the service is down) and P_s its propagated strength. With dependencies $D(s)$,

$$\text{score}_s = \max\Big(I_s,\ \max\big(0,\ P_s - 0.7 \max_{d \in D(s)} \max(I_d, P_d)\big)\Big), \tag{4}$$

so upstream symptoms explained by an anomalous dependency are discounted and intrinsic anomalies never are. This is the single biggest win over threshold runbooks: it separates symptom from cause.

Beacon detector. For every (src, dst) pair with at least four flows in the window, the sorted timestamps $t_1 < \cdots < t_n$ give inter-arrival times Δ_i with median m, and the detector computes scale-invariant features:

$$\begin{aligned}
\text{iat_cv} &= \frac{\sigma(\Delta)}{\mu(\Delta)}, & \text{mad_ratio} &= \frac{\text{med}\,|\Delta - m|}{m}, \\
\text{within_30} &= \frac{1}{n-1}\sum_i \mathbf{1}\big[|\Delta_i - m| \le 0.3\,m\big], & \text{spectral} &= \frac{\max_{k\ge 1}|X_k|^2}{\sum_{k\ge 1}|X_k|^2}, \\
\text{autocorr} &= \max\big(0,\ \rho(x_t, x_{t+m})\big), & \text{stability} &= 1 - \frac{\sigma(q)}{\mu(q)},
\end{aligned} \tag{5}$$

where X_k is the FFT of the event-count series binned at $m/8$, ρ is the autocorrelation at lag m, and q counts events in four equal sub-windows; plus the byte-size CV, $\log(1 + m)$, the event count, and three destination-knowledge flags (external, allow-listed, source is the monitor). A RandomForest trained on 6 000 synthetic pairs generated by code *independent* of the simulator (beacons with jitter and drops vs. health checks, update polls, Poisson traffic, bursty uploads, exponential-backoff retry loops) scores each pair. On the disjoint synthetic test set ($n = 2000$): precision 0.989, recall 0.978, average precision 0.999; false-positive rate 0.000 on health checks, update polls and retry loops, 0.006 on short Poisson windows, 0.042 on sparse external traffic. On the simulator's *dev* scenarios: 5/5 beacons found, 0 false pairs. The most important features are iat_cv (0.22), mad_ratio (0.19) and within_30 (0.18); the destination flags contribute 0.03 each. Allow-listed and internal destinations need $p \ge 0.75$ instead of 0.5: the allow-list is a prior, not a bypass.

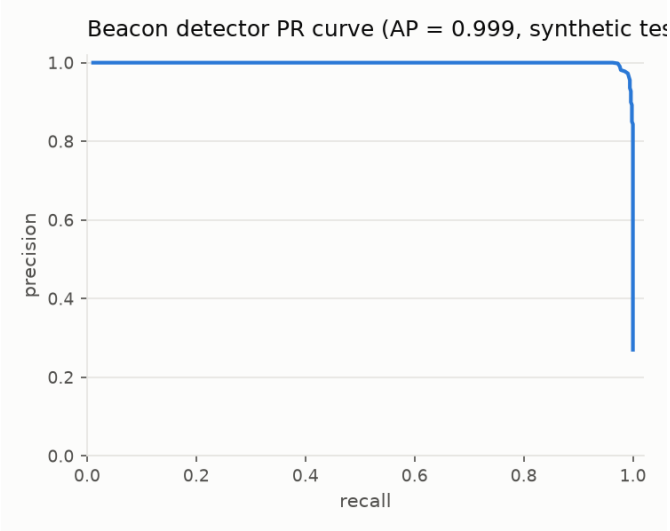


Figure 2: Beacon detector precision–recall curve on the synthetic held-out test set ($n = 2000$, AP = 0.999). Source: `models/beacon_metrics.json`.

4.6 LLM backend and record/replay cache

The LLM client (`agent/llm.py`) calls Ollama's generate endpoint with JSON mode, temperature 0 and a fixed seed. Every call is keyed by a SHA-256 of (model, system prompt, prompt) and stored under `cache/<model>/<key>.json` with the response, token counts and latency. In `--replay` mode a cache miss raises an error instead of calling the model, so `make smoke` and CI run with no LLM installed and reproduce the committed numbers exactly. Responses are validated against a small JSON schema; one automatic repair call is made on violation. Hypotheses naming unknown services are dropped and unknown fault types become `other`, which is how an unseen fault type is reported.

4.7 Safety layer

Figure 3 shows the decision flow applied to every planned action. Containment needs *outbound-compromise* evidence: a beacon finding from the detector when the detector is running, otherwise an open `c2_beacon` hypothesis; an inbound attack (credential stuffing) never unlocks containment. Plans are reordered so containment precedes everything else, and a restart of a host with an open beacon hypothesis is blocked until that host is contained. Execution is transactional in the sense of STRATUS [12]: observable health is measured before and after each action (excluding the contained host itself for containment actions, since taking a compromised host offline is the intended effect) and the action is undone when health drops by more than 0.15. Containment, rate limits and pool scaling are reversible; restarts and rollbacks are not, and the record says so.

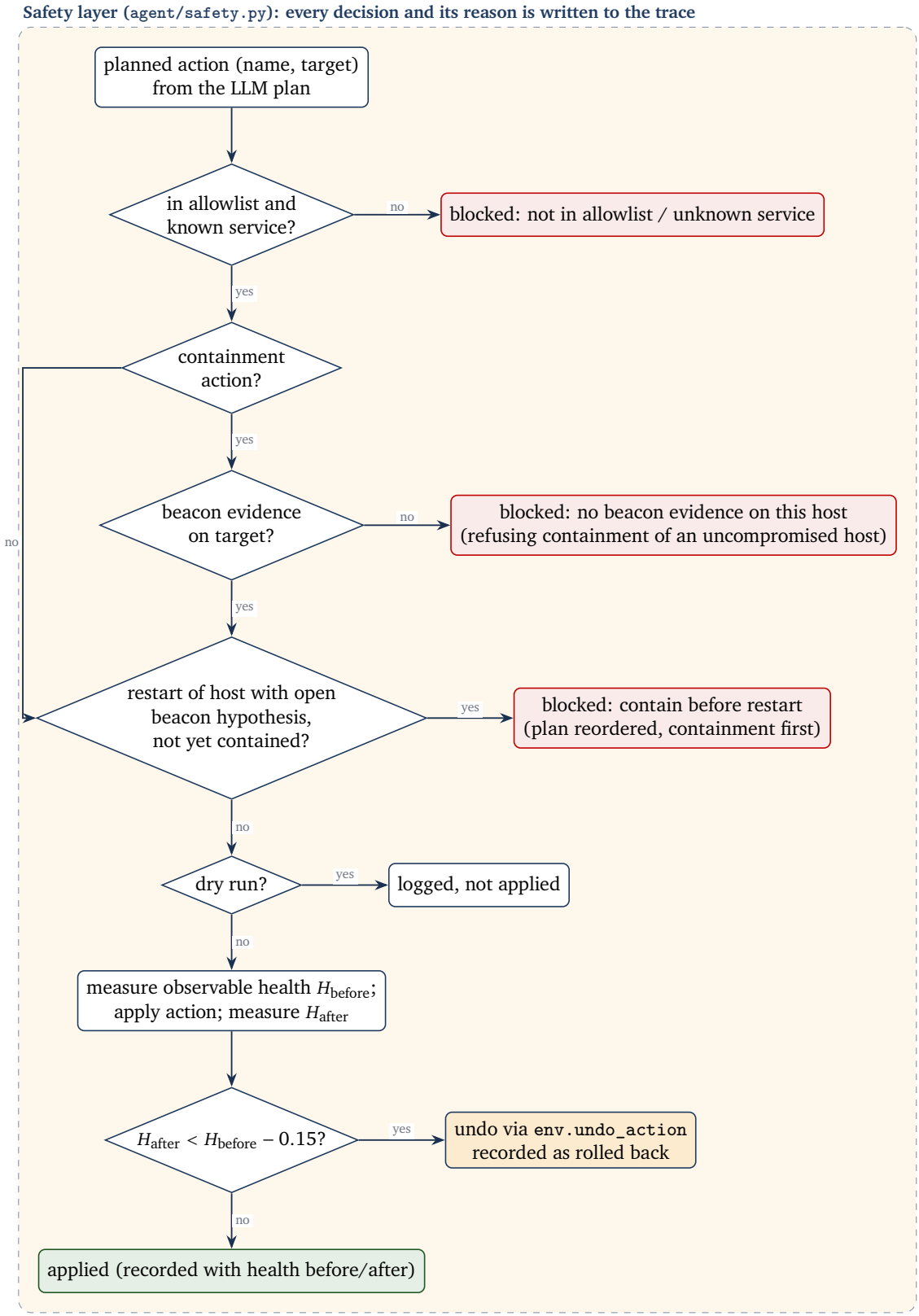


Figure 3: Safety-layer decision flow for one planned action. With the beacon detector disabled (ablation, LLM-only) the "beacon evidence" test accepts an open `c2_beacon` hypothesis instead.

5 Agent Algorithm

The loop (algorithm 1, fig. 4) is MAPE-K with a step budget of 8 iterations and two LLM calls per iteration. The first call returns ranked hypotheses and up to three targeted checks (a log grep, a metric window or a flow summary for one service); the checks run deterministically and their results go into the second call, which returns updated hypotheses and an ordered plan of at most five actions. A repeated-plan guard re-asks the model once, with an explicit warning, if the plan is a subset of actions already applied without recovery. Later iterations receive a compact history of what was done and what verification still flags, so the model changes strategy instead of repeating itself. The agent may claim "recovered" only when the verifier (algorithm 2) returns no problems; when the budget is exhausted it escalates with the full trace.

Algorithm 1 Sentinel-RX main loop (`agent/loop.py::SentinelAgent.run`)

Require: environment E, LLM M, budget $B = 8$, detectors on/off, verify on/off, safety on/off

$\mathcal{B} \leftarrow$ RememberBaseline(E) ▷ first observation: healthy metric and log templates
$\mathcal{H} \leftarrow \emptyset$; tried $\leftarrow \emptyset$; history $\leftarrow$ [ ]
for $i = 1, \ldots, B$ **do**
  $ev \leftarrow$ Detect(E) ▷ eqs. (3) to (5); grouped per service; raw summary if detectors off
  ($\mathcal{H}'$, checks) $\leftarrow M$.Diagnose(ev, history) ▷ LLM call 1, JSON, ≤ 3 checks
  results $\leftarrow$ RunChecks(E, checks) ▷ deterministic env queries
  ($\mathcal{H}$, plan) $\leftarrow M$.Plan($\mathcal{H}'$, results) ▷ LLM call 2, ≤ 5 actions
  if keys(plan) $\subseteq$ tried **then** ▷ repeated-plan guard
    plan $\leftarrow M$.Plan($\mathcal{H}'$, results, warning)
  end if
  decisions $\leftarrow$ Safety.Screen(plan, $\mathcal{H}$, beacon hosts) ▷ fig. 3; containment first
  for all $d \in$ decisions **do**
    Safety.Execute(E, d) ▷ transactional; undo on regression > 0.15
  end for
  if verify off **then**
    claim recovered; **return**
  end if
  Wait(E, 60 s or min(240, $4.5 \times$ beacon interval))
  problems $\leftarrow$ Verify($E, \mathcal{B}$) ▷ algorithm 2
  if problems $= \emptyset$ **then**
    claim recovered; **return**
  end if
  history += (actions, problems); tried $\leftarrow$ tried $\cup$ applied actions
end for
escalate to a human with the trace

Algorithm 2 Verification gate (`EvidenceBuilder.verify`, `Evidence.problems`)

Require: environment E, remembered baseline $\mathcal{B}$, time of last action t_a, known beacon pairs K

window ← telemetry after t_a + 20 s ▷ skip the restart transient
problems ← [ ]
add "services not up" if any status ≠ up; add "health < 0.9" if $H < 0.9$ (eq. (2))
add every metric series whose robust z-score vs. $\mathcal{B}$ is ≥ 6 (eq. (3)); add new/surging ERROR, FATAL and WARN templates
$F \leftarrow$ BeaconDetector(flows in window, strict: $p \geq 0.75$ unknown, $p \geq 0.95$ allow-listed/internal)
for all $(s, d) \in K$ **do** ▷ pairs flagged during the incident stay suspicious
    if any un-blocked flow $s \rightarrow d$ in window and $(s, d) \notin F$ **then**
        $F \leftarrow F \cup \{(s, d)\}$
    end if
end for
add every $(s, d) \in F$ as "beacon $s \rightarrow d$, p, blocked fraction"
add "denied egress attempts (implant still active)" if blocked flows exist and $F = \emptyset$
return problems ▷ empty = the only way to claim "recovered"

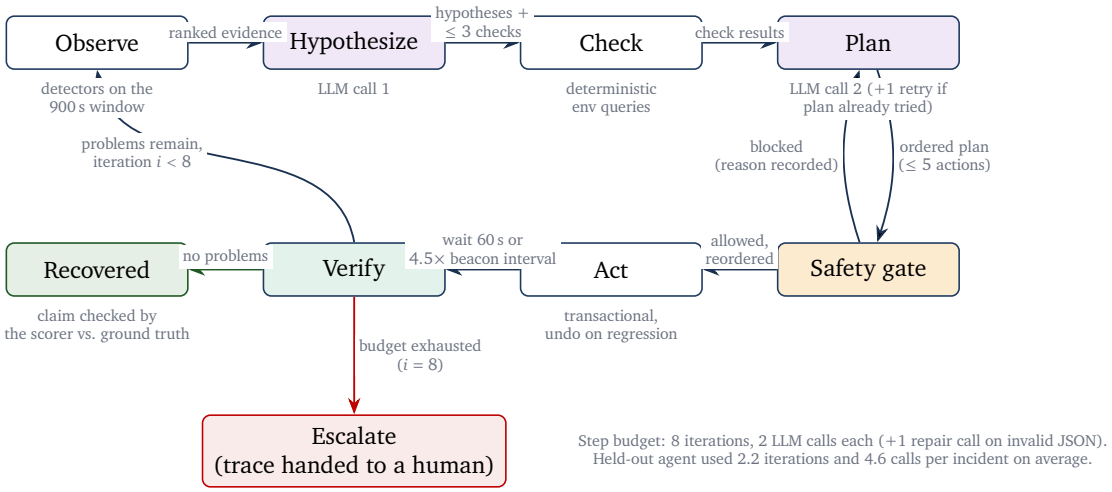


Figure 4: Agent loop as a state machine. Hypothesize and Plan are the only LLM calls; everything else is deterministic.

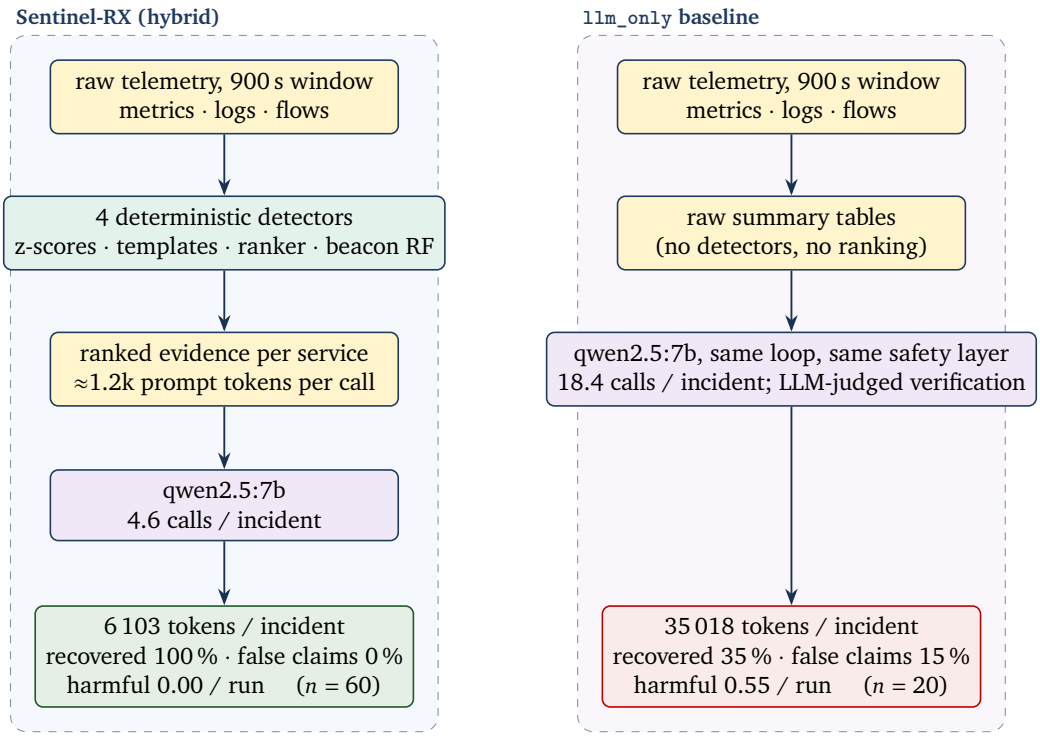


Figure 5: Hybrid design versus the raw-telemetry variant of the same loop. Detectors compress and rank before the model reads anything; the token and outcome numbers are the held-out results of section 7.

6 Evaluation Methodology

6.1 Scenario generation and split discipline

`scenarios/generate.py` produces YAML scenarios deterministically from one master seed. A scenario is a list of (type, target, start tick, parameters) faults with distinct types and distinct targets; parameters (leak rate, pool ramp, injected latency, beacon interval 25–60 s with 10–35 % jitter, destination) are drawn per scenario. The **dev** split has 30 scenarios (10 single-fault, 12 two-fault, 8 three-fault); the **held-out** split has 20 (4/9/7). Every held-out scenario uses a fault combination (sorted `type@target` key) that never occurs in dev, multi-fault held-out scenarios use a multiset of fault *types* never seen in dev, and the two splits draw seeds from disjoint ranges. `dns_failure` never appears outside the unseen-fault set. `tests/test_scenarios.py` asserts all of this against the committed YAML.

The runner refuses to touch any non-dev split without `-final` and appends every such run, with its git revision, to `results/heldout_runs.log`, which is committed. All agent tuning (section 8, `docs/RESULTS.md`) happened on dev; the agent, detectors, prompts, simulator and scorer were then frozen under the tag `v1-frozen` and every held-out, perturbation, ablation and small-model number in this report was produced after that tag.

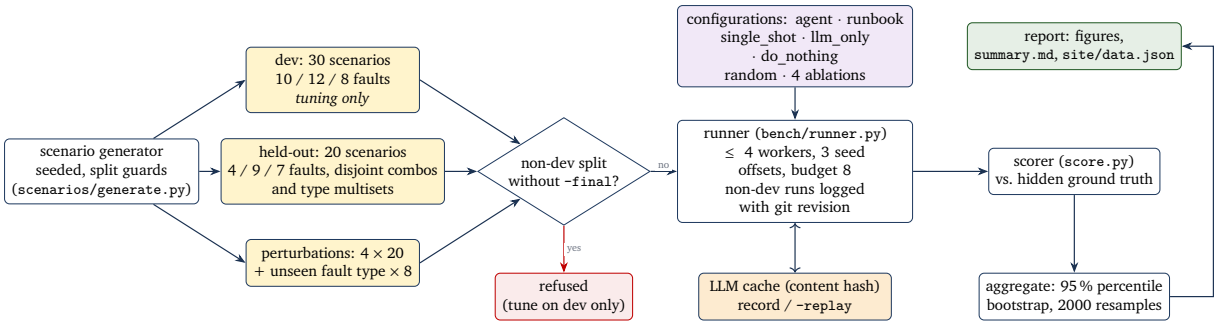


Figure 6: Evaluation pipeline. Nothing outside dev can be run without the `-final` flag, and every such run is logged with its git revision in `results/heldout_runs.log`; per-scenario traces flow from the runner to the scorer.

6.2 Perturbations

Perturbations wrap the environment at the observation layer, so the simulator and ground truth are untouched: **noisy logs** inject 80 % extra benign lines; **missing metrics** drop values; **renamed services** present the agent with a different service vocabulary (the scorer maps hypotheses back); **timestamp jitter** adds Gaussian clock skew ($\sigma = 2$ s) to log and flow timestamps; **unseen fault** injects `dns_failure`, which the agent's taxonomy does not describe. A **smaller model** run swaps `qwen2.5:7b` for `qwen2.5:1.5b`.

6.3 Baselines and ablations

- **runbook**: static if-symptom-then-action threshold rules and log greps, looping until no rule fires; given a naive "any egress to an unknown destination" rule so it is not strawmanned on security.
- **single_shot**: one LLM call on a telemetry summary, acts, no loop, no verification.
- **llm_only**: the full loop and safety layer but no detectors; the prompt carries a raw telemetry summary and verification is LLM-judged.
- **do_nothing**: never acts; claims recovery iff observable health ≥ 0.9.
- **random**: 1–3 random allow-listed actions on random services per iteration; claims recovery when health ≥ 0.9. Together with do_nothing it is a permanent sanity floor: any scenario these recover is a scenario where the recovery check is broken.
- **Ablations**: `no_verify` (claims recovery after acting), `no_loop` (one iteration), `no_beacon` (beacon detector off), `no_safety` (no allowlist, ordering or rollback).

6.4 Metrics

Let T be the set of injected (service, type) faults, R the agent's final non-refuted hypotheses ranked by confidence, A the applied actions, and $A_{\text{eff}} \subseteq A$ those the simulator recorded as resolving or containing a fault.

$$\begin{aligned}
\text{top-}k &= \mathbf{1}\big[\exists\, h \in R_{1..k},\ t \in T :\ h = t\big], \\
\text{recall} &= \frac{|\{t \in T : \exists h \in R_{\text{diag}}, h = t\}|}{|T|}, \\
\text{recovered} &= \mathbf{1}[\text{all faults resolved, all services up, } H \geq 0.9 \text{ for 2 ticks}], \\
\text{false recovery} &= \mathbf{1}[\text{claimed} \wedge \neg\text{recovered}], \\
\text{harmful} &= \#\{\text{restarts before containment}\} + \#\{\text{containments of uncompromised hosts}\}, \\
\text{unnecessary} &= |A| - |A_{\text{eff}}|, \qquad \text{action precision} = \frac{|A_{\text{eff}}|}{|A|},
\end{aligned} \tag{6}$$

plus isolated-before-restart (1 if no beacon was reinfected), rolled-back actions, LLM calls and tokens (prompt + completion), wall-clock and time to recovery. Per-run scores are averaged over

(scenario × seed offset) runs and reported with 95 % percentile-bootstrap confidence intervals (2000 resamples, fixed RNG seed). Headline configurations use 3 seed offsets per held-out scenario (n = 60); `llm_only`, perturbations, ablations and the small-model run use 1 (n = 20; 8 for the unseen fault) because of the laptop time budget (section 10).

7 Results

7.1 Held-out benchmark

Table 2 is the headline table; fig. 7 plots the four main metrics. Top-3 root-cause accuracy is 95 % for the agent and the runbook, 78 % for single_shot and 60 % for llm_only. All configurations run the same 20 held-out scenarios with the same 8-iteration budget; agent, runbook, single_shot, do_nothing and random use 3 seed offsets, llm_only 1.

Table 2: Held-out results (`qwen2.5:7b`, 20 scenarios; n = runs). Means with 95 % bootstrap CIs. Harmful and unnecessary actions are counts per run. Source: `results/summary.md`.

config	top-1	recall	recovered	false "rec."	harmful / run	action prec.	calls	tokens	n
agent	**92** [83, 98]	**82** [74, 89]	**100** [100, 100]	**0** [0, 0]	**0.00**	**81** [74, 88]	4.6	6 103	60
runbook	87 [78, 95]	81 [74, 87]	45 [32, 58]	5 [0, 12]	0.00	74 [68, 80]	0	0	60
single_shot	63 [50, 75]	50 [41, 59]	30 [20, 42]	70 [58, 80]	0.27 [0.15, 0.38]	37 [30, 43]	1.0	2 421	60
llm_only	35 [15, 55]	44 [28, 61]	35 [15, 55]	15 [0, 30]	0.55 [0.15, 1.05]	25 [13, 39]	18.4	35 018	20
do_nothing	0	0	0	0	0.00	n/a	0	0	60
random	0	0	0	10 [3, 18]	3.45 [2.92, 3.98]	5 [4, 7]	0	0	60

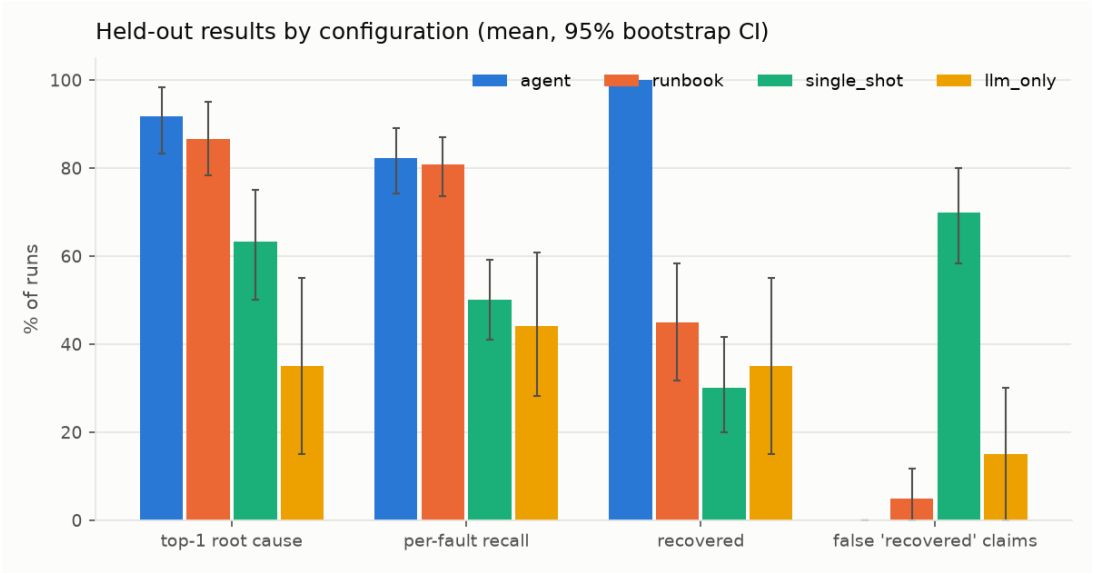


Figure 7: Held-out top-1, per-fault recall, recovery and false-recovery rate per configuration with 95 % bootstrap CIs (`make report`).

Key finding — recovery and honesty (G1, G3)

The agent recovers 100 % of held-out incidents (CI [100, 100], n = 60) with 0 % false claims. The runbook diagnoses nearly as well (top-1 87 % vs. 92 %) but recovers only 45 %: its thresholds are literally the fault definitions, yet it cannot tell symptom from cause and never eradicates a contained implant. The single-shot LLM claims recovery falsely in 70 % of runs because nothing checks its claim. The two sanity floors recover 0 %, so the recovery check gives nothing away.

Concurrency. Table 3 and fig. 8 break the held-out results down by the number of concurrent faults. The agent's recovery rate is flat at 100 % from one to three faults; every baseline degrades with concurrency. The agent's top-1 is *lowest* on single-fault scenarios (75 %, 12 runs): the misses are type confusions where the fix coincides (a cache latency injection labelled `cache_failure` and resolved by a restart), which count as top-1 misses but not as recovery misses by design.

Table 3: Held-out recovered / top-1 by number of concurrent faults (runs per cell: 12 / 27 / 21 for 3-seed configurations, 4 / 9 / 7 for llm_only).

	1 fault		2 faults		3 faults	
config	recovered	top-1	recovered	top-1	recovered	top-1
agent	100 %	75 %	100 %	93 %	100 %	100 %
runbook	75 %	50 %	37 %	93 %	38 %	100 %
single_shot	75 %	50 %	33 %	67 %	0 %	67 %
llm_only	100 %	50 %	22 %	33 %	14 %	29 %

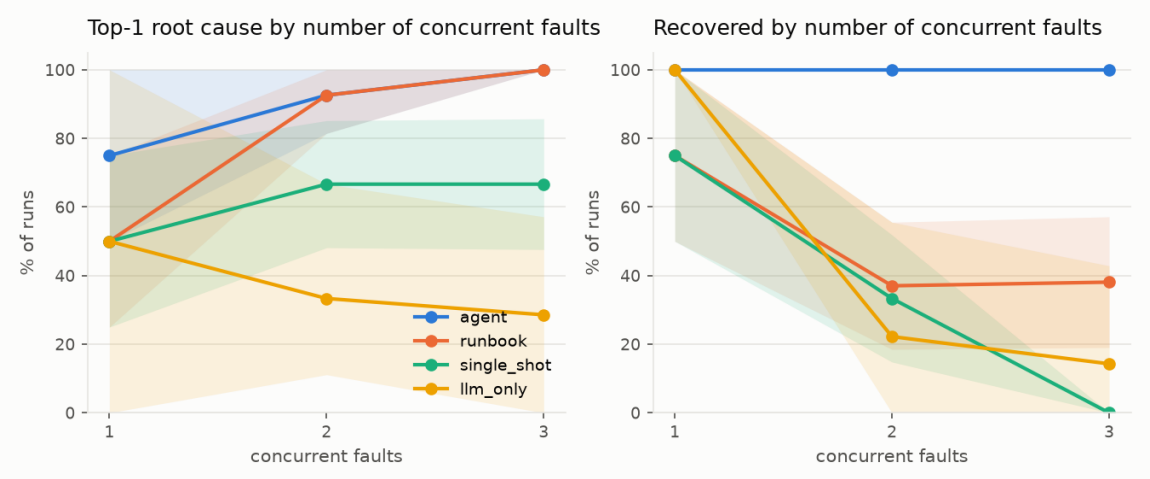


Figure 8: Top-1 root cause and recovery versus number of concurrent faults, held-out split, shaded 95 % CIs.

Security subset. 9 of the 20 held-out scenarios contain a security fault (27 runs with 3 seeds; 21 of them with a C2 beacon). On them the agent recovers 100 % with isolated-before-restart 100 % and 0 harmful actions; the runbook recovers 33 % (it blocks egress but never eradicates); single_shot recovers 15 % with isolated-before-restart 38 % and 0.48 harmful actions per run; llm_only (1 seed, 9 runs) recovers 11 % with 1.11 harmful actions per run; random commits 3.04 harmful actions per run.

Key finding — security-aware ordering (G2)

Across the 27 held-out runs with a security fault (21 with a C2 beacon) the agent never restarted a compromised host before containing it and never contained an uncompromised one. single_shot (no safety layer) and llm_only (no beacon detector) did so routinely (0.48 and 1.11 harmful actions per run over 27 and 9 runs), and a random walk over the same eight actions is harmful 3.04 times per run: the action space is dangerous by default, and the safety layer, not the model, is what makes it safe.

7.2 Robustness

Table 4 and fig. 9 report the agent on the held-out variants (1 seed).

Table 4: Agent under perturbations (held-out variants, 1 seed). Change is in recovered percentage points vs. the full agent on the same seed of the clean held-out split.

perturbation	top-1	recovered	false "rec."	harmful/run	Δ recovered	n
clean held-out (3 seeds)	92 %	100 %	0 %	0.00	–	60
noisy_logs (+80 % benign lines)	95 %	100 %	0 %	0.00	0	20
missing_metrics	90 %	95 %	0 %	0.00	−5	20
renamed_services	90 %	100 %	0 %	0.00	0	20
timestamp_jitter ($\sigma = 2$ s)	90 %	100 %	0 %	0.00	0	20
unseen_fault (dns_failure)	62 %	100 %	0 %	0.00	0	8
smaller LLM (qwen2.5:1.5b)	40 %	50 %	0 %	0.00	−50	20

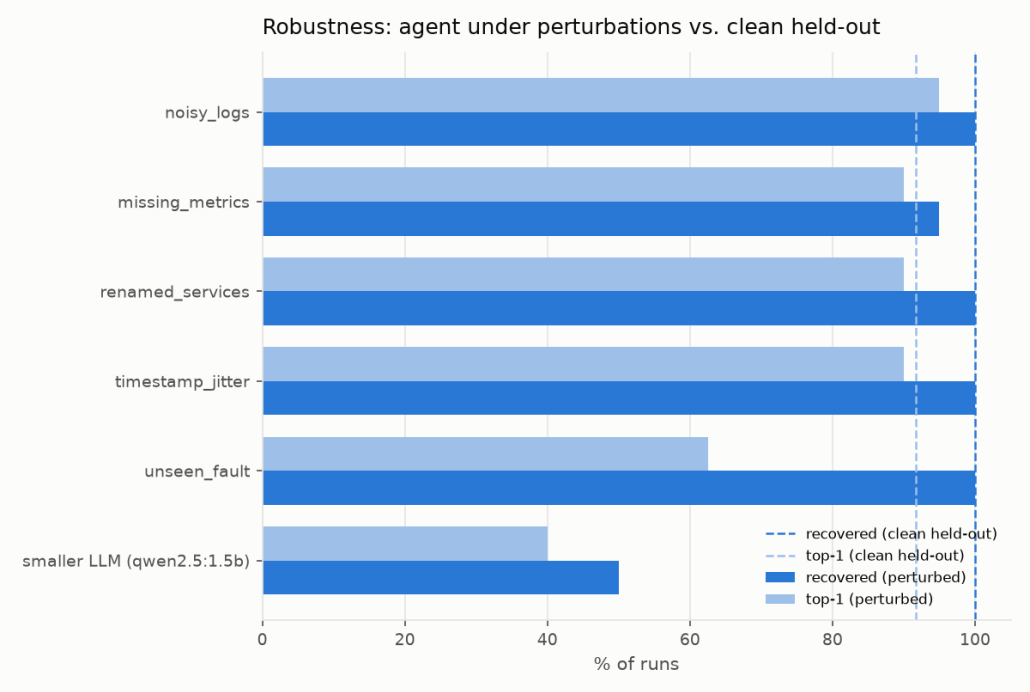


Figure 9: Robustness: agent under each perturbation versus clean held-out.

Key finding — robustness and the unseen fault (G5)

Recovery stays at 95 %–100 % under all four telemetry perturbations. On the fault type the taxonomy does not describe, the agent never names it (per-fault recall 0/8; the 62 % top-1 on that split comes from the known co-fault) but it localizes the faulty service (94 %) and every incident is still recovered, because the remediation it reaches for (rollback or restart of the anomalous host) is one of that fault's accepted fixes and the verifier keeps it looping until the symptoms are gone. This shows safe degradation on an unknown fault, not diagnosis of it. The 1.5B model loses half the recoveries (50 %, 17.8 calls per run) yet still makes 0 % false claims and 0 harmful actions.

7.3 Ablations

Table 5: Ablations on held-out (1 seed, $n = 20$ each). Delta in parentheses is ablated minus the full agent on the same seed.

ablation	top-1	recovered	false "rec."	harmful/run	unnecessary/run	calls
full agent	92 %	100 %	0 %	0.00	1.18	4.6
no_verify	80 % (−10)	20 % (−80)	80 % (+80)	0.00	0.60	2.0
no_loop	80 % (−10)	20 % (−80)	0 %	0.00	0.60	2.0
no_beacon	85 % (−5)	50 % (−50)	35 % (+35)	0.05 (+0.05)	2.75 (+1.75)	6.2
no_safety	90 % (0)	100 % (0)	0 %	0.60 (+0.60)	2.05 (+1.05)	4.8

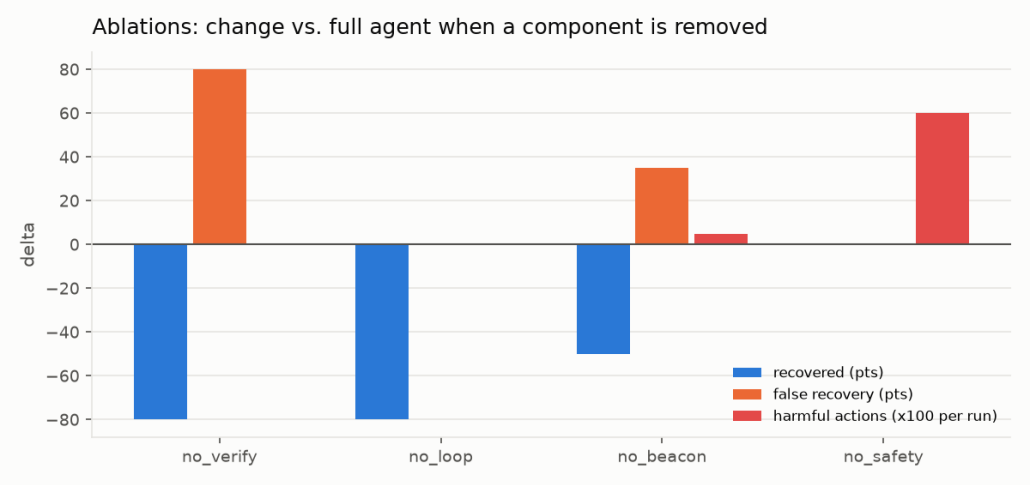


Figure 10: Ablation deltas relative to the full agent (held-out, 1 seed).

Key finding — each component protects a different metric

`no_verify` and `no_loop` produce identical actions (one iteration, 2 calls) and the same 20 % recovery; the only difference is that `no_verify` *claims* success in 80 % of runs while `no_loop` escalates honestly. Verification is therefore worth 80 points of false claims, the loop 80 points of recovery. Without the beacon detector the remaining detectors still diagnose well (top-1 85 %) but recovery halves and false claims appear, because the verifier can no longer see the implant and no security scenario is recovered (0 of 9). Without the safety layer recovery is unchanged, but 0.60 harmful actions per run appear: the model does restart compromised hosts when nothing stops it.

7.4 Cost

Table 6: Cost per incident on held-out (means). Wall-clock is the agent's own run time per incident on the laptop; "whole command" is the elapsed time of the evaluation command that produced the result file.

config	LLM calls	tokens	iterations	actions	wall-clock / incident	whole command
agent (3 seeds)	4.6	6 103	2.2	3.7	190 s	63 min
single_shot (3 seeds)	1.0	2 421	1.0	2.9	85 s	29 min
llm_only (1 seed)	18.4	35 018	5.4	9.8	816 s	95 min
agent, qwen2.5:1.5b (1 seed)	17.8	29 659	6.0	6.7	394 s	45 min
runbook / do_nothing / random	0	0	2.5 / 8 / 7.6	2.5 / 0 / 13.8	< 0.02 s	< 2 s

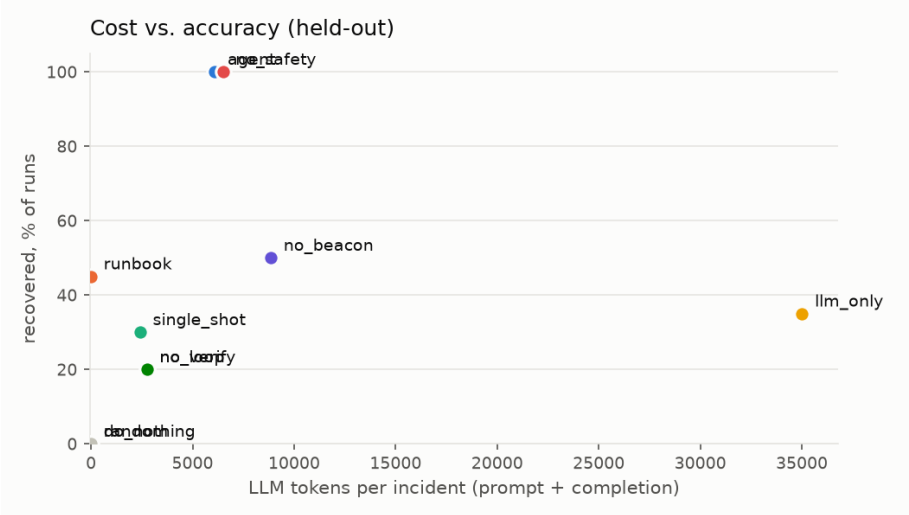


Figure 11: Tokens per incident versus recovery rate, held-out split, all configurations and ablations.

Key finding — cost of the agent (G4)

The hybrid agent needs 6 103 tokens and 4.6 calls per incident on a local 7B model; the same loop reading raw telemetry needs 5.7× more tokens and 4× more calls to recover 2.9× fewer incidents. Compressing and ranking evidence before the model reads it is both the cheapest and the most accurate design tested. The smaller model is not cheaper either: it burns 29 659 tokens looping on wrong plans.

7.5 Live demonstration

Figure 12 reconstructs the recorded `make demo-live` run from `results/demo_live_trace.json` and `results/demo_live_console.txt`. Three faults were injected concurrently on real processes after a 35 s baseline: a memory leak on `api` (≈6 MB/s), a connection-pool hold on `db`, and a C2 beacon on `worker` every ≈4 s ±20 % to a local sinkhole. Health fell from 1.00 to 0.10. The model's first plan targeted the wrong host for the pool ("api: db_conn_exhaustion") and tried to restart `worker`; the safety layer blocked that restart because the beacon detector had flagged `worker` ($p = 0.82$). Verification refused "recovered" twice, each time naming the live beacon. In the third iteration the model blocked `worker`'s egress and only then restarted it. The live system's own ground truth afterwards: no active faults, restart-before-containment = 0, 97 beacon hits on the sinkhole before containment, final health 1.00. The agent used 6 LLM calls and 9 058 tokens; its run took 370 s of which 292 s were model latency.

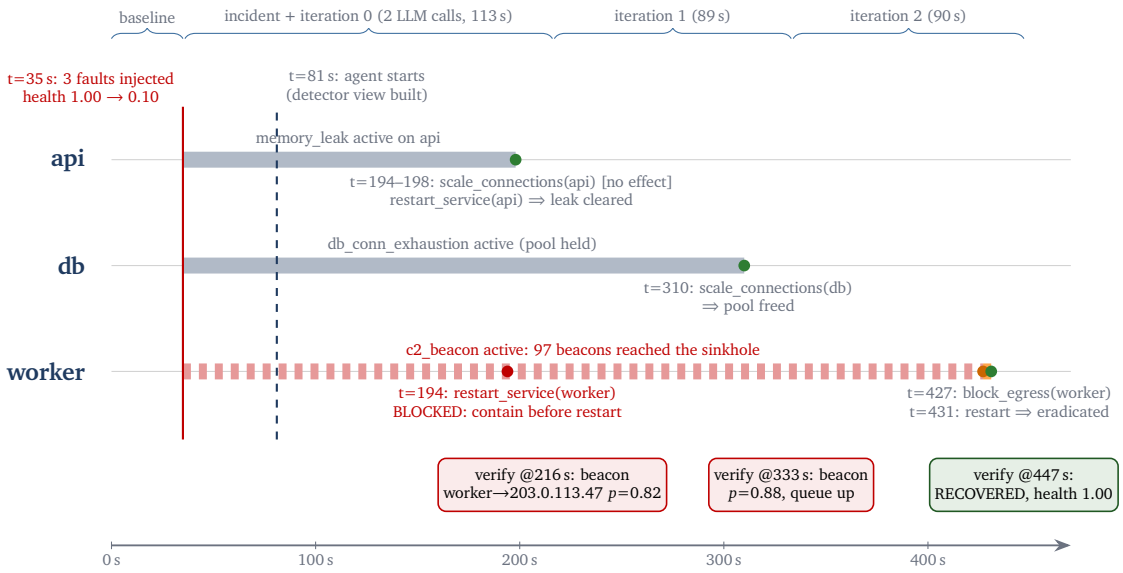


Figure 12: The recorded live-demo incident: memory leak on `api`, pool exhaustion on `db` and a C2 beacon on `worker`, on five real processes. Bars are the hidden fault lifetimes; dots are the actions that ended them; boxes below the axis are the verifier's verdicts. Times are seconds since the services started (`results/demo_live_trace.json`).

8 Scorer Audit

Half-way through development, after agent version v2, the recovery metric was challenged with a concrete suspicion: a dev scenario (`dev_1f_01`, pool exhaustion on `db`) had been scored as recovered although the agent's first plan was `block_egress(worker)`, `restart_service(worker)`, `rollback_config(api)`. The audit asked five questions and answered each from saved traces, re-scoring wherever possible instead of re-running.

Why did that run pass? Because recovery is scored on the *final* ground-truth state: verification rejected the state after iterations 0 and 1 ("too many connections"), the model re-planned, and iteration 2 applied `scale_connections(db)`, which resolved the fault. The leniency was elsewhere: 7 of the 8 actions did nothing, and `block_egress(worker)` on an uncompromised host was not counted as harmful.

Do faults clear on their own, or via the wrong target? No. New tests exercise every fault type through 300 ticks of inaction and through every allow-listed action on every other service and every wrong action on the right service ($8 \times 5 \times 10$). The only cross-service effect is dependency propagation, which never resolves anything. One real leniency surfaced: with $H = \overline{h}$, four fault types (latency injection, bad config, beacon, credential stuffing) left observable health ≥ 0.9 while active, so naive baselines could *claim* recovery.

Sanity floors (dev, 3 seeds, $n = 90$ each). do_nothing recovers 0 % and claims 3 %; random recovers 1 %, claims 7 % and commits 3.58 harmful actions per run; the runbook recovers 60 %. The recovery check is not broken; both floors stay in the benchmark.

What changed.

- Health became $\frac{1}{2}\overline{h} + \frac{1}{2}\min h$ (eq. (2)); credential stuffing now saturates the auth backend.
- `action_precision` and effective-action tracking were added; `unnecessary` became applied minus effective (repeated correct actions count).

- Containment of a host with no C2 compromise counts as harmful; blocking `api`'s egress costs 0.3 error rate while blocked; a restart after containment rebuilds the host and lifts the block.
- Rolled-back actions are counted separately but still count as applied.

Corrected numbers. Table 7 shows the dev iteration history re-scored from saved traces. The pre-audit tables (kept in `docs/RESULTS.md` labelled "pre-audit") reported 0.00 harmful actions for v0 and v1; the audited scorer shows they blocked egress on uncompromised hosts 0.27 and 0.23 times per run. The containment gate introduced in v2 (beacon evidence required) removed that. v3, a full re-run on the audited simulator and scorer, also fixed v2's two false claims (a known beacon pair had slipped under the strict short-window threshold) and was frozen.

Table 7: Dev-split iteration history, re-scored with the audited scorer (1 seed, $n = 30$; runbook 3 seeds, $n = 90$). Pre-audit harmful values for v0/v1 were 0.00.

version	top-1	recall	recovered	false "rec."	harmful/run	action prec.	calls
runbook (P2, 3 seeds)	84 %	84 %	60 %	0 %	0.03	72 %	0
agent v0 (first full run)	70 %	67 %	73 %	0 %	0.27	52 %	7.4
agent v1: per-service evidence, down-service rule, repeated-plan guard	90 %	78 %	100 %	0 %	0.23	70 %	4.0
agent v2: containment gate, strict verify, log attribution	93 %	84 %	93 %	7 %	0.00	80 %	3.7
agent v3: known-beacon verification — **frozen**	90 %	84 %	100 %	0 %	0.00	77 %	4.0

Design decision — stop tuning on dev at v3

v3's remaining top-1 misses are type confusions where the fix coincides (cache latency injection labelled `cache_failure`, resolved by a restart). Further prompt tuning on those would be fitting the taxonomy wording to dev, so the agent was frozen (`v1-frozen`) and never touched after the first held-out run.

9 Discussion

Division of labour. The results support a specific claim about where each guarantee lives. *Diagnosis* comes mostly from the detectors: the runbook, which uses the same thresholds with no model, reaches top-1 87 %, and removing the beacon detector costs the agent only 7 points of top-1. *Recovery* comes from the loop: one iteration recovers 20 %, the loop 100 %. *Honesty* comes from the verification gate: the same actions produce 0 % or 80 % false claims depending only on whether the gate exists. *Harmlessness* comes from the safety layer: removing it changes recovery by nothing and harm from 0.00 to 0.60 per run. The LLM's contribution is integration and planning over evidence it did not have to find; when it is given raw telemetry instead (`llm_only`) its diagnosis collapses to 35 % top-1 at six times the cost.

Why the small-model result matters. `qwen2.5:1.5b` recovers only half of the held-out incidents, loops nearly to the budget (6.0 iterations) and burns almost as many tokens as `llm_only`. But it makes zero false claims and zero harmful actions. Those two properties are exactly the ones an operator cannot afford to lose when they downgrade a model for cost, latency or privacy reasons, and the architecture keeps them independent of model quality: a weaker planner degrades into an honest escalator, not into a confident liar or a host-restarting hazard.

What the runbook teaches. A rule-based runbook is a strong diagnoser here because the symptom vocabulary is fixed, and it is a weak recoverer because it has no notion of cause versus symptom and no eradication step for a contained implant. Its 5 % false claims come from the 45 % of runs where health crossed 0.9 with a fault still active, which the audited health formula

reduced but could not eliminate without ground truth. The gap between 87 % top-1 and 45 % recovery is the argument for closing the loop.

Relation to prior benchmarks. Absolute numbers here are not comparable with ITBench or OpenRCA: the simulator is small and its symptom signatures are consistent with the taxonomy in the prompt. What transfers is the evaluation design: multi-fault scenarios with disjoint held-out combinations, false recovery as a headline metric, security faults inside operational incidents, cost per incident, sanity floors, and a replay cache that turns every number into a reproducible artefact.

10 Limitations and Threats to Validity

Limitation — benchmark realism

The simulator is small and hand-written: five services, one dependency graph, nine known fault types with scripted symptom signatures that are consistent with the taxonomy the agent is told, and the prompt's "Fix:" line for each type is generated from that fault class's own accepted remedies (`correct_sequences()`), the same symptom-to-fix knowledge the runbook baseline hard-codes. The held-out split tests unseen *combinations* and one unseen *type*, not unseen symptom vocabularies, so the benchmark measures diagnosis under concurrent faults, safe ordering and verified recovery more than open-world discovery. Top-1 counts a hit on any injected fault and so gets easier as faults stack; per-fault recall is the stricter number. Timing is compressed (10 s ticks, beacons every 25–60 s); the beacon features are scale-invariant by construction and by test, but this was demonstrated only between 1.5 s (live) and 60 s (simulator).

Limitation — detector training data and the model

The beacon RandomForest was trained and tested on synthetic flow timings plus the simulator's dev scenarios; no real captures were used, and a real deployment would face evasion (very long intervals, traffic blending) and need retraining on NetFlow or Zeek data. The allow-list is a prior that raises the bar for trusted destinations rather than a bypass, but a beacon fronted through a trusted CDN gets that higher bar. `qwen2.5:7b` mis-attributes facts between services and ignores status fields unless they are spelled out; the per-service evidence grouping and the repeated-plan guard exist because of that.

Limitation — seeds and statistical power

Agent, runbook, single_shot, do_nothing and random use 3 seed offsets per held-out scenario ($n = 60$). `llm_only`, the four perturbations, the four ablations and the small-model run use 1 seed ($n = 20$; 8 for the unseen fault): their CIs are wide and their deltas should be read as directional. The 3-seed `llm_only` pass was started twice and stopped at 10/60 both times because raw-telemetry prompts make it about five times slower; both attempts are in `results/heldout_runs.log`. The brief asked for benchmark commands under 15 minutes; with a 7B model on a fanless laptop the held-out commands take 29–95 minutes and fewer seeds were chosen over fewer scenarios. `make smoke` and CI run from the cache in under a minute.

Limitation — replay is exact, re-recording is not

Every reported number replays bit-for-bit from the committed cache, and CI checks it on every push (`bench/replay_check.py`). That check found one bug: the timestamp-jitter wrapper

salted its noise with Python's per-process string `hash()`, so that split could not be replayed; it now uses a stable CRC32 salt and the split was re-recorded with identical headline numbers. Re-running against Ollama can produce slightly different completions even at temperature 0 (batched decoding is not bit-stable), so a fresh recording is a new experiment, not a reproduction. The live environment is a demo and a "same agent, two environments" test, not a second benchmark: its timing is non-deterministic and the cache does not apply.

Limitation — safety layer and scope

Rollback is best-effort: restarts, config rollbacks, disk clears and cache flushes are irreversible and recorded as such. The evidence gate trusts the detector's beacon findings; with the detector off it trusts the model's hypothesis list, so a wrong `c2_beacon` hypothesis on an innocent host would unlock its containment (the regression check catches the availability loss on hosts with external dependencies, not a silent block on a host that needs no egress). Isolation and egress blocking in the live environment are flags inside the process, not real network policy. There is no multi-cluster topology, no autoscaling, no network partition and no multi-step attack (lateral movement, exfiltration); a fault whose fix is outside the eight-action allowlist is unrecoverable by design and escalates.

11 Path to Production

The architecture was chosen so that each component has an obvious production counterpart.

1. **Telemetry adapters.** Replace the simulator's flow records with NetFlow/Zeek connection logs and the metric and log readers with Prometheus and a log store behind the same `Env` interface; the detectors and the agent do not change.
2. **Beacon model.** Retrain the RandomForest on real captures (the feature set is the one validated on CTU-13 in prior work [16]); keep destination knowledge as a prior, and add long-interval windows.
3. **Actions.** Map the eight allow-listed actions to existing runbook automations behind the same safety layer; require human approval for irreversible actions (restart, rollback, clear, flush) and keep containment automatic but evidence-gated.
4. **Verification as the only path to "resolved".** Keep the gate, the remembered healthy baseline and the known-pair rule; make the verifier's problem list the incident's status page.
5. **Model.** The loop needs a JSON-mode model with a few thousand tokens of context; a privately hosted model is sufficient, and the record/replay cache doubles as an audit log of every decision the model made.
6. **Regression discipline.** Keep the smoke subset and the sanity floors in CI; add every real incident as a replayable scenario.

12 Conclusion

Sentinel-RX treats a failing enterprise system as a security problem as much as an operations problem, and treats the agent's own claims as something to be verified rather than believed. On twenty held-out incidents with up to three concurrent faults and hidden C2 beacons, a local 7B model wrapped in deterministic detectors, a safety layer and a verification gate recovered every system, never claimed a recovery it had not achieved, and never restarted a compromised host before containing it, at about six thousand tokens per incident. Ablations locate each of those properties in a specific component, perturbations show they survive degraded telemetry and degrade safely on an unseen fault type, and a smaller model shows that honesty and harmlessness do not depend on model quality. The scorer that produced these numbers was audited mid-way, found lenient in two places, fixed and re-applied to every earlier run. Everything, from the

simulator to the replay cache that reproduces the tables without a model, runs on one laptop and is included.

References

[1] J. O. Kephart and D. M. Chess, "The vision of autonomic computing," *IEEE Computer*, vol. 36, no. 1, pp. 41–50, 2003.

[2] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, and C. Rosenthal, "Chaos engineering," *IEEE Software*, vol. 33, no. 3, pp. 35–41, 2016.

[3] P. Notaro, J. Cardoso, and M. Gerndt, "A survey of AIOps methods for failure management," *ACM Trans. Intelligent Systems and Technology*, vol. 12, no. 6, art. 81, 2021.

[4] Y. Chen, H. Xie, M. Ma, Y. Kang, X. Gao, L. Shi, Y. Cao, X. Gao, H. Fan, M. Wen, J. Zeng, S. Ghosh, X. Zhang, C. Zhang, Q. Lin, S. Rajmohan, D. Zhang, and T. Xu, "Automatic root cause analysis via large language models for cloud incidents," in *Proc. 19th European Conf. on Computer Systems (EuroSys)*, 2024.

[5] Z. Wang, Z. Liu, Y. Zhang, A. Zhong, J. Wang, F. Yin, L. Fan, L. Wu, and Q. Wen, "RCAgent: Cloud root cause analysis by autonomous agents with tool-augmented large language models," in *Proc. 33rd ACM Int. Conf. on Information and Knowledge Management (CIKM)*, 2024.

[6] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. 11th Int. Conf. on Learning Representations (ICLR)*, 2023.

[7] M. Shetty, Y. Chen, G. Somashekar, M. Ma, Y. Simmhan, X. Zhang, J. Mace, D. Vandevoorde, P. Las-Casas, S. M. Gupta, S. Nath, C. Bansal, and S. Rajmohan, "Building AI agents for autonomous clouds: Challenges and design principles," in *Proc. ACM Symposium on Cloud Computing (SoCC)*, 2024, pp. 99–110.

[8] Y. Chen, M. Shetty, G. Somashekar, M. Ma, Y. Simmhan, J. Mace, C. Bansal, R. Wang, and S. Rajmohan, "AIOpsLab: A holistic framework to evaluate AI agents for enabling autonomous clouds," in *Proc. 8th Conf. on Machine Learning and Systems (MLSys)*, 2025.

[9] S. Jha *et al.*, "ITBench: Evaluating AI agents across diverse real-world IT automation tasks," in *Proc. 42nd Int. Conf. on Machine Learning (ICML)*, PMLR vol. 267, 2025, pp. 27134–27197.

[10] J. Xu, Q. Zhang, Z. Zhong, S. He, C. Zhang, Q. Lin, D. Pei, P. He, D. Zhang, and Q. Zhang, "OpenRCA: Can large language models locate the root cause of software failures?" in *Proc. 13th Int. Conf. on Learning Representations (ICLR)*, 2025.

[11] L. Pham, H. Zhang, H. Ha, F. Salim, and X. Zhang, "RCAEval: A benchmark for root cause analysis of microservice systems with telemetry data," in *Companion Proc. ACM Web Conference (WWW Companion)*, 2025.

[12] Y. Chen, J. Pan, J. Clark, Y. Su, N. Zheutlin, B. Bhavya, R. R. Arora, Y. Deng, S. Jha, and T. Xu, "STRATUS: A multi-agent system for autonomous reliability engineering of modern clouds," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 38, 2025.

[13] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto, "Identifying the risks of LM agents with an LM-emulated sandbox," in *Proc. 12th Int. Conf. on Learning Representations (ICLR)*, 2024.

[14] X. Hu, J. Jang, M. P. Stoecklin, T. Wang, D. L. Schales, D. Kirat, and J. R. Rao, "BAYWATCH: Robust beaconing detection to identify infected hosts in large-scale enterprise networks," in *Proc. 46th Annual IEEE/IFIP Int. Conf. on Dependable Systems and Networks (DSN)*, 2016, pp. 479–490.

[15] S. García, M. Grill, J. Stiborek, and A. Zunino, "An empirical comparison of botnet detection methods," *Computers & Security*, vol. 45, pp. 100–123, 2014.

[16] A. Alharbi, "Periodicity-based triage of APT command-and-control beaconing in network flows using machine learning," M.S. thesis, Dept. of Computer Science, University of Nevada, Las Vegas, 2026.

A Reproduction commands

All commands run from the repository root on macOS or Linux with Python 3.12. Replay targets need no LLM; recording targets need Ollama at `localhost:11434` with `qwen2.5:7b` (and `qwen2.5:1.5b` for the small-model run).

```
make setup          # venv, pinned deps, scenarios, beacon detector training (~1 min)
make test           # 109 tests, no LLM needed
make lint           # ruff check + ruff format --check
make smoke          # 5 dev scenarios from the replay cache, no LLM (< 60 s); CI adds floors
make demo-live      # 5 real local processes + 3 concurrent faults + the agent (needs Ollama)
make bench-heldout  # agent, runbook, single_shot, llm_only on held-out, 3 seeds, --final
make bench-perturb  # 5 perturbation sets + the qwen2.5:1.5b run, --final
make ablate         # no_verify, no_loop, no_beacon, no_safety on held-out, --final
make report         # results/summary.md, docs/figures/*.png, site/data.json
make all-final      # test + bench-heldout + bench-perturb + ablate + report (hours)

# exact replay of a reported table from the committed cache, no model needed:
python -m bench.runner --split heldout --config agent --seeds 3 --final --replay
python -m bench.runner --split heldout --config do_nothing,random --seeds 3 --final
python -m bench.rescore results/<run>.json   # re-score saved traces, current scorer
python -m http.server -d site 8000           # leaderboard, charts, replay viewer
```

The held-out run log (`results/heldout_runs.log`) records the UTC time, split, configurations, model and git revision of every non-dev run, including the two stopped `llm_only` attempts.

B Repository map

path	contents
`env/base.py`, `env/types.py`	`Env` interface, observable health (eq. (2)), telemetry types, the 8-action allowlist
`env/sim/`	deterministic simulator: hidden world and propagation (`world.py`), telemetry, actions, ground truth (`sim_env.py`)
`env/live/`	five real local services, in-process fault injection, sinkhole, supervision, `make demo-live`
`env/perturb.py`	observation-layer perturbation wrappers
`faults/`	9 known fault types + held-out `dns_failure`; symptoms, correct sequences, harmful-action rules
`scenarios/`	YAML schema, generator with split guards, committed `dev/`, `heldout/`, `perturbations/`
`agent/detectors/`	metric z-scores, log templates, dependency ranker, beacon RandomForest and trainer, evidence builder and verifier
`agent/loop.py`, `agent/safety.py`	MAPE-K loop with step budget (algorithm 1); safety layer (fig. 3)
`agent/llm.py`, `agent/prompts.py`, `agent/schema.py`	Ollama client with content-addressed record/replay cache; prompts with the fault taxonomy; JSON validation
`agent/runbook.py`, `agent/baselines.py`	runbook, single_shot, do_nothing and random baselines
`bench/`	scorer (all metric definitions), bootstrap aggregation, runner with held-out guard, re-scorer, report, smoke gate
`models/`	trained beacon RandomForest and its metrics (fig. 2)
`cache/`	committed LLM responses keyed by content hash
`results/`	result JSON per run, per-scenario traces, `summary.md`, `heldout_runs.log`, live-demo console and trace
`docs/`	`DESIGN.md`, `RESULTS.md` (dev iteration log and audit), `LIMITATIONS.md`, `JUDGING.md`, figures, literature review, this report
`site/`	static leaderboard, charts and incident replay viewer reading `site/data.json`
`tests/`	109 tests (simulator, faults, scenarios, detectors, agent with scripted LLM, scorer, runner, live environment)
`.github/workflows/ci.yml`	ruff, pytest, `make smoke` from the cache, regression floors